

TASAS BACH DOCUMENTACIÓN

DOCUMENTACIÓN SOBRE EL CSS QUE HEMOS USADO Y EL JAVASCRIPT

TASAS-BACH-STYLE.CSS

Esta documentación detalla la arquitectura de estilos utilizada en el archivo `tasas-bach-style.css`, diseñada para ser usada junto a la hoja de estilos `base base-style.css`.

1. SISTEMA DE DISEÑO

Hemos usado variables para los colores porque, no solo es más cómodo, sino que, si el día de mañana nos piden cambiar el azul por otro tono, lo cambiamos en el código y se actualizaría en toda la página sin tener que ir buscando línea por línea:

VARIABLE	VALOR HEX	USO
--color-principal	#f6eddc	Fondos de sección.
--color-enlace	#e3e5d7	Textos destacados y fondos de contraste suave.
--color-contraste-1	#a5c8ca	Bordes decorativos y botones.
--color-contraste-2	#586875	Títulos secundarios.
--color-oscuro	#2c343a	Fondo del formulario y encabezados.
--color-texto	#333	Cuerpo de texto

2. ARQUITECTURA DE SECCIONES

A. PAGE-INTRO

Hemos metido un degradado suave que va desde el color de contraste hasta el fondo crema para que la entrada a la página no sea tan brusca.

B. CONTACTO-CONTAINER

Para esta parte, usamos un grid dividido en dos. Le dimos un poco más de espacio a la derecha, donde se encuentra el formulario, porque, al ser donde el usuario tiene que escribir, no queríamos que se viera todo muy junto y de forma agobiante.

C. CONTACTO-FORM

Hemos puesto el formulario con el fondo oscuro para que destaque bien sobre el resto de la información. Nos pareció que los inputs con fondo semitransparente le pueden dar un toque más moderno.

D. SERVICIOS-GRID

En la parte de abajo, he organizado el contenido alternando el texto y la imagen en dos columnas. Me parece que de esta forma la lectura es mucho menos pesada y la página queda más equilibrada visualmente.

Para las imágenes de los alumnos, he usado la propiedad `object-fit: cover`. Lo hice porque me di cuenta de que, al cambiar el tamaño de la ventana, las fotos se estiraban de forma rara; con este ajuste, me aseguro de que mantengan siempre sus proporciones y se vean bien sin deformarse, sin importar el tamaño de la pantalla.

3. RESPONSIVE DESIGN

Hemos configurado el responsive para que la web no se rompa en pantallas pequeñas. El cambio más importante lo pusimos a los 992px; ahí los grids pasan a ser una sola columna para que la información se lea de forma lógica y ordenada en el móvil, sin que nada se vea minúsculo

4. GUÍA DE MANTENIMIENTO

A los botones les hemos metido una transición rápida de 0.3 segundos. Es un detalle pequeño, pero ese efecto de que el botón se eleva al pasar el ratón hace que la web se sienta mucho más profesional al interactuar con ella

TASAS-BACH.JS

La parte de JavaScript de esta sección es fundamental, ya que es la que se encarga de que el formulario de solicitud realmente "haga algo" a ojos del usuario. Al igual que con el contador de caracteres, he buscado que la página sea interactiva y profesional, gestionando el envío de datos de forma dinámica.

Para que el script funcione correctamente, se utiliza una constante llamada `formulario` que busca el elemento `form` dentro de la clase `contacto-form` usando `document.querySelector()`. A partir de ahí, todo ocurre dentro de una función principal:

- **formulario.addEventListener('submit', function(e):** Esta función se activa en cuanto el usuario pulsa el botón de enviar. Lo primero que hace es usar `e.preventDefault()`, que sirve para que la página no se recargue automáticamente, permitiéndonos manejar el proceso nosotros mismos con JavaScript.
- **Captura de datos:** Se crea un objeto llamado `datos` que guarda lo que el usuario ha escrito en cada campo (nombre, email, especialidad y mensaje). Para la especialidad, he usado un `querySelectorAll` con un `[1]` para pillar el segundo input de texto, ya que es donde se pregunta por el instrumento o danza.
- **Validación:** He incluido un pequeño "if" que comprueba si los campos de nombre o email están vacíos. Si el usuario se olvida de ellos, salta un alert avisando de que son obligatorios y detiene el proceso.
- **Feedback visual (El "Envío"):** Como todavía no tenemos un servidor real conectado, he programado una simulación. Cuando le das a enviar, el botón cambia su texto a "Enviando..." y se vuelve un poco transparente para que el usuario vea que la web está trabajando.
- **setTimeout() y respuesta final:** Después de 1.5 segundos (para que parezca que realmente se está mandando algo por internet), el código borra el formulario y lo sustituye por un mensaje de agradecimiento personalizado. Este mensaje usa el nombre y el email que puso el usuario para que quede mucho más profesional, además de incluir un botón para volver atrás que recarga la página con `location.reload()`.

Al final, este JavaScript es bastante directo pero muy efectivo, ya que garantiza que el usuario reciba una respuesta inmediata de la escuela tras su consulta, priorizando la funcionalidad y la buena imagen de la institución.